

SS

SupplySync

Enterprise Supply Chain Management System

Complete Technical Documentation & API Reference

VERSION

2.0 Enterprise

BACKEND

.NET 10 | ASP.NET Core

FRONTEND

React 18 | TypeScript

DATABASE

SQL Server | EF Core

Admin

Vendor

Procurement Officer

Warehouse Manager

Finance Officer

Compliance Officer

Document Date: May 2026

Organization: SupplySync Technologies Pvt. Ltd.

Confidential — Internal Use Only

Table of Contents

1.	Executive Summary	3
2.	System Architecture	4
	2.1 Technology Stack	4
	2.2 Project Structure	4
3.	Data Models & Database Schema	5
	3.1 Core Entities	5
	3.2 Entity Relationships	5
4.	User Roles & Permissions	6
5.	Application Workflow	7
	5.1 End-to-End Process Flow	7
	5.2 Vendor Onboarding Flow	7
	5.3 Procurement Flow	7
6.	API Reference	8
	6.1 Authentication APIs	8
	6.2 Vendor APIs	9
	6.3 Contract APIs	9
	6.4 Purchase Order APIs	9
	6.5 Goods Receipt APIs	10
	6.6 Inventory APIs	10
	6.7 Invoice & Payment APIs	11
	6.8 Compliance APIs	11
	6.9 Reports APIs	11
7.	Frontend Architecture	12
8.	Notification System	13
9.	Security & Authentication	13
10.	Testing	14
11.	Deployment & Configuration	15

1 Executive Summary

SupplySync is a full-stack, enterprise-grade Supply Chain Management System designed to digitalise and streamline the end-to-end procurement lifecycle. Built on .NET 10 (ASP.NET Core Web API) with a React 18 TypeScript frontend, the platform serves six distinct roles: Admin, Vendor, Procurement Officer, Warehouse Manager, Finance Officer, and Compliance Officer.

The system covers every stage of the procurement cycle — from vendor self-registration and approval, through contract negotiation, purchase-order issuance, goods receipt verification, invoice submission and three-way matching, payment processing, inventory management, compliance auditing, and real-time notification delivery.

Metric	Value
Total API Endpoints	45+
User Roles Supported	6
Core Business Modules	10
Database Entities	13
Frontend Dashboards	6 role-specific
Test Coverage (NUnit)	19 test files / 60+ tests
Authentication	JWT Bearer — 8 hr expiry
Notification Triggers	12 event types

Key Capabilities

- Vendor self-registration portal with document upload and approval workflow
- Contract management with Draft → Active → Closed lifecycle
- Purchase Order creation with multi-item support and vendor notification
- Goods Receipt with binary Accept/Reject decision and automatic inventory update
- Three-way invoice matching (PO ↔ GR ↔ Invoice) before Finance approval
- Payment processing with full audit trail and reference tracking
- Role-based compliance auditing across all entities
- Real-time notification system with polling-based delivery
- Six fully-featured role-specific dashboards with live statistics
- Comprehensive reporting: vendor performance, spending, delays, compliance

2 System Architecture

2.1 Technology Stack

Layer	Technology	Version	Purpose
Backend Runtime	.NET / ASP.NET Core	10.0	Web API framework
ORM	Entity Framework Core	10.0	Database access & migrations
Database	Microsoft SQL Server	2022	Relational data store
Authentication	ASP.NET Core Identity + JWT	Latest	Auth & role management
Object Mapping	AutoMapper	12.x	DTO ↔ Model mapping
Frontend	React + TypeScript	18 / 5.x	SPA user interface
Build Tool	Vite	5.x	Frontend bundler
Styling	Tailwind CSS	3.x	Utility-first CSS
HTTP Client	Axios	1.x	API communication
State / Auth	React Context API	18	Global auth & theme state
Testing	NUnit + Moq	4.x / 4.x	Unit & integration tests
API Docs	Swagger / OpenAPI	3.0	Interactive API explorer

2.2 Project Structure

The solution follows a clean layered architecture pattern with strict separation of concerns:

Layer	Folder	Responsibility
Presentation	Controllers/	HTTP request handling, routing, auth guards
Service Layer	Services/	Business logic, orchestration, notifications
Repository Layer	Repositories/	Data access abstraction, EF Core queries
Domain	Models/	Entity definitions, enums, domain rules
Data Transfer	DTOs/	Request/Response contracts, validation
Interfaces	Interfaces/	Contracts for DI, testability
Data Access	Data/AppDbContext.cs	EF Core DbContext, model configuration
Cross-cutting	Helpers/ / Mappings/	JWT generation, AutoMapper profiles
Frontend — API	src/api/	Axios instance, typed API modules
Frontend — Pages	src/pages/	Role-specific dashboard components
Frontend — Context	src/context/	Auth, Theme, Notification providers

Frontend — Types	src/types/index.ts	TypeScript interfaces for all entities
------------------	--------------------	--

3 Data Models & Database Schema

3.1 Core Entities

Entity	Purpose	Key Fields	Notes
ApplicationUser	Extends IdentityUser	Id, FullName, IsActive, CreatedAt	Auth identity for all roles
Vendor	Vendor profile	Id, VendorCode, CompanyName, ContactEmail, TaxNumber, LicenseNumber, Status, UserId	Links to ApplicationUser
Contract	Procurement contract	Id, ContractNumber, VendorId, StartDate, EndDate, PaymentTerms, Status	Draft→Active→Closed
PurchaseOrder	Purchase order	Id, PONumber, VendorId, ContractId, ExpectedDeliveryDate, Status, Items[]	Created→Sent→Delivered
POItem	Line item on PO	Id, PurchaseOrderId, ItemName, Quantity, UnitPrice	Child of PurchaseOrder
GoodsReceipt	Delivery verification	Id, GRNumber, PurchaseOrderId, Status, ReceivedDate, Items[]	Accept / Reject
GoodsReceiptItem	GR line item	Id, GoodsReceiptId, ItemName, ReceivedQuantity, Condition	Good / Damaged
InventoryItem	Stock record	Id, ItemName, SKU, QuantityInStock, Unit, LastUpdated	Auto-updated on GR
ItemIssue	Stock issuance	Id, InventoryItemId, QuantityIssued, IssuedTo, IssueDate	Reduces stock
Invoice	Vendor invoice	Id, InvoiceNumber, VendorId, PurchaseOrderId, GoodsReceiptId, TotalAmount, Status	3-way match
Payment	Payment record	Id, InvoiceId, AmountPaid, PaymentReference, PaymentDate	1-to-1 with Invoice
ComplianceCheck	Audit record	Id, EntityType, EntityId, Status, Remarks, CheckedByUserId	Pass / Fail
Notification	In-app notification	Id, UserId, Message, IsRead, Type, CreatedAt	12 event types

3.2 Entity Relationships

From	Relationship	To	Cascade
Vendor	1 — Many	Contract	NoAction
Vendor	1 — Many	PurchaseOrder	NoAction
Vendor	1 — Many	Invoice	NoAction
Contract	1 — Many	PurchaseOrder	NoAction
PurchaseOrder	1 — Many	POItem	Cascade

PurchaseOrder	1 — Many	GoodsReceipt	NoAction
PurchaseOrder	1 — Many	Invoice	NoAction
GoodsReceipt	1 — Many	GoodsReceiptItem	Cascade
InventoryItem	1 — Many	ItemIssue	Cascade
Invoice	1 — 1	Payment	Cascade
ApplicationUser	1 — Many	Vendor	Cascade

4 User Roles & Permissions

Role	Created By	Dashboard	Key Permissions
Admin	System seed	/admin	Manage users, toggle active/inactive, reset passwords, change roles, view a
Vendor	Self-register	/vendor	View own POs, mark as Delivered, submit invoices, view contracts, reapply
Procurement Officer	Admin	/procurement	Approve/reject vendors, create contracts, create & cancel POs, view deliver
Warehouse Manager	Admin	/warehouse	Receive goods (Accept/Reject), manage inventory, issue items, view all GR
Finance Officer	Admin	/finance	Review & approve/reject invoices, process payments, view spending reports
Compliance Officer	Admin	/compliance	Perform compliance checks on any entity (Pass/Fail), view audit log, monito

All routes are protected by **JWT Bearer authentication**. Each controller action is decorated with **[Authorize(Roles = "...")]** to enforce role-based access control at the API level. The frontend uses a **ProtectedRoute** component that reads the user's role from **AuthContext** and redirects unauthorised users to a 403 page.

5 Application Workflow

5.1 End-to-End Process Flow

Step	Actor	Action	System Effect
1	Vendor	Self-registers on portal	Account created, status = Pending. Procurement Officers notified.
2	Procurement Officer	Reviews & approves vendor	Vendor status → Approved. Vendor receives notification.
3	Procurement Officer	Creates Contract (Draft)	Contract saved. Vendor notified. Status = Draft.
4	Procurement Officer	Activates Contract	Status → Active. PO creation now allowed.
5	Procurement Officer	Creates Purchase Order	PO created, status = Sent. Vendor notified via notification.
6	Vendor	Marks PO as Delivered	PO status → Delivered. Warehouse Manager can now create GR.
7	Warehouse Manager	Receives goods (Accept / Reject)	Accept: GR created, PO stays Delivered, inventory updated. Reject: GR created as Rejected, PO reverts to Sent.
8	Vendor	Submits invoice (links PO + GR)	Invoice created (Submitted). Finance Officers notified.
9	Finance Officer	Reviews invoice (Approve / Reject)	Approved: Invoice status → Approved. Vendor notified. Rejected: Invoice rejected with reason. Vendor notified.
10	Finance Officer	Processes payment	Payment record created. Vendor notified with reference number.
11	Compliance Officer	Performs compliance check on any entity	Pass/Fail recorded. On Fail: Admin & Procurement notified.

5.2 Vendor Rejection & Re-delivery Flow

- Warehouse Manager rejects delivery → GR status = Rejected, PO reverts to Sent
- Vendor sees 'Mark Delivered' button re-enabled on their dashboard
- Vendor ships again and marks PO as Delivered once more
- Warehouse Manager sees PO in Incoming Deliveries (rejected GR does not block)
- New GR created as Accepted → inventory updated → vendor can submit fresh invoice

5.3 Three-Way Invoice Matching

Before a Finance Officer can approve an invoice, the system validates:

- ✓ The invoice is linked to a valid Purchase Order belonging to the vendor
- ✓ The Goods Receipt linked to the invoice has status = Accepted (not Rejected or Pending)
- ✓ No non-rejected duplicate invoice exists for the same PO + GR combination
- ✓ The invoice status is currently Submitted (not already Approved or Rejected)

6 API Reference

Base URL: **http://localhost:5000/api** | All endpoints require **Authorization: Bearer <token>** except login and vendor-register.

6.1 Authentication — /api/auth

Method	Endpoint	Role(s)	Description
POST	/register	Admin	Create internal user with specific role
POST	/vendor-register	Public	Vendor self-registration with company details
POST	/login	Public	Login — returns JWT token, role, vendorId

6.2 Vendor Management — /api/vendor

Method	Endpoint	Role(s)	Description
GET	/	Admin, Procurement, Compliance, Finance	Get all vendors
GET	/[{id}]	Admin, Procurement, Compliance, Finance, Vendor	Get vendor by ID
GET	/my-profile	Vendor	Get own vendor profile
PUT	/[{id}]/status	ProcurementOfficer	Approve or reject vendor with optional reason
PUT	/[{id}]/suspend	Admin	Suspend vendor account
PUT	/reapply	Vendor	Reapply after rejection with updated documents

6.3 Contract Management — /api/contract

Method	Endpoint	Role(s)	Description
GET	/	Admin, Procurement, Compliance, Finance	Get all contracts
GET	/[{id}]	Authenticated	Get contract by ID
GET	/vendor/{vendorId}	Authenticated	Get contracts for a vendor

POST	/	ProcurementOfficer	Create new contract (status = Draft)
PUT	/ {id} / activate	ProcurementOfficer	Activate draft contract
PUT	/ {id} / close	ProcurementOfficer	Close active contract

6.4 Purchase Orders — /api/purchaseorder

Method	Endpoint	Role(s)	Description
GET	/	Admin, Procurement, Warehouse, Compliance, Finance	Get all POs
GET	/ {id}	Authenticated	Get PO by ID with items
GET	/ vendor / {vendorId}	Vendor, Procurement, Admin	Get POs for a vendor
POST	/	ProcurementOfficer	Create PO (requires active contract)
PUT	/ {id} / status	Vendor, Procurement	Update PO status (e.g. Delivered)
PUT	/ {id} / cancel	ProcurementOfficer	Cancel PO if not yet delivered

6.5 Goods Receipts — /api/goodsreceipt

Method	Endpoint	Role(s)	Description
GET	/	Admin, Warehouse, Procurement, Compliance, Finance	Get all goods receipts
GET	/id	Admin, Warehouse, Procurement, Compliance, Finance	Get GR by ID
GET	/po/{poId}	Admin, Warehouse, Procurement, Compliance, Finance, Vendor	Get GRs for a PO
POST	/	WarehouseManager	Create GR (Accept / Reject). Updates inventory on Accept. Reverts PO to Sent on Reject.

6.6 Inventory — /api/inventory

Method	Endpoint	Role(s)	Description
GET	/	Admin, Warehouse, Procurement, Compliance	Get all inventory items
GET	/id	Admin, Warehouse, Procurement, Compliance	Get inventory item by ID
POST	/	WarehouseManager	Create or update inventory item (adds quantity if exists)
POST	/issue	WarehouseManager	Issue item to department/employee. Triggers low-stock alert if qty ≤ 10
GET	/issues	Admin, Warehouse, Compliance	Get all item issue records
GET	/id/issues	Admin, Warehouse, Compliance	Get issue history for specific item

6.7 Invoices & Payments — /api/invoice

Method	Endpoint	Role(s)	Description
GET	/	Admin, Finance, Compliance	Get all invoices
GET	/id	Admin, Finance, Compliance, Vendor	Get invoice by ID with payment details

GET	/vendor/{vendorId}	Admin, Finance, Vendor	Get invoices for a vendor
POST	/	Vendor	Submit invoice (links PO + GR + amount)
PUT	/[{id}]/review	FinanceOfficer	Approve or reject invoice. GR must be Accepted.
POST	/payment	FinanceOfficer	Process payment for approved invoice

6.8 Compliance — /api/compliance

Method	Endpoint	Role(s)	Description
GET	/	Admin, Compliance	Get all compliance checks
GET	/entity	Admin, Compliance	Get checks by entity type & ID
GET	/failed	Admin, Compliance	Get all failed checks
POST	/	ComplianceOfficer	Record Pass/Fail check. On Fail: notifies Admin & Procurement.

6.9 Reports — /api/reports

Method	Endpoint	Role(s)	Description
GET	/vendor-performance	Admin, Procurement, Compliance	PO counts, invoice counts, total spend per vendor
GET	/inventory-summary	Admin, Warehouse, Compliance	Stock levels and total issued per item
GET	/procurement-spending	Admin, Finance, Procurement	Approved invoices with payment details & total
GET	/delivery-delays	Admin, Procurement, Compliance	POs past expected delivery date with days delayed
GET	/item-issues	Admin, Warehouse, Compliance	All item issue records ordered by date
GET	/compliance-summary	Admin, Compliance	Pass/Fail/Pending counts + 10 most recent failures

6.10 Notifications — /api/notification

Method	Endpoint	Role(s)	Description
--------	----------	---------	-------------

GET	/	Authenticated	Get all notifications for logged-in user
GET	/unread-count	Authenticated	Get count of unread notifications
PUT	/id/read	Authenticated	Mark single notification as read
PUT	/read-all	Authenticated	Mark all notifications as read

6.11 Admin — /api/admin

Method	Endpoint	Role(s)	Description
GET	/users	Admin	Get all users with roles
PUT	/users/id/toggle-active	Admin	Activate or deactivate user account
PUT	/users/id/reset-password	Admin	Reset user password
PUT	/users/id/change-role	Admin	Change user role

7 Frontend Architecture

The frontend is a React 18 + TypeScript Single Page Application built with Vite. Each role gets a completely isolated dashboard with its own navigation and feature set. The sidebar adapts dynamically based on the authenticated user's role.

Dashboard	Route	Tabs / Sections
Admin	/admin	Users, Vendors, Create User, Reports
Vendor	/vendor	Overview, Purchase Orders, Invoices, Contracts
Procurement Officer	/procurement	Vendors (approval), Contracts, Purchase Orders, Delayed POs
Warehouse Manager	/warehouse	Incoming Deliveries, Inventory, Goods Receipts, Item Issues
Finance Officer	/finance	Invoices (review + pay), Payment History, Spending Report
Compliance Officer	/compliance	Overview, Vendors, Contracts, POs, Inventory Issues, Invoices, Audit Log

Context Providers

Context	State	Key Methods
AuthContext	user, isAuthenticated, isLoading	login(userData), logout()
ThemeContext	isDark	toggleTheme() — persists to localStorage
NotificationContext	unread (count)	refreshUnread() — polled every 15 seconds

Shared Components

Component	Props	Usage
StatCard	title, value, icon, color, subtitle	Dashboard KPI cards
Badge	status: string	Colour-coded status pills across all tables
Modal	isOpen, onClose, title, size	All create/review/detail dialogs
LoadingSpinner	—	Data loading states
Layout	children	Sidebar + Topbar + Footer wrapper
ProtectedRoute	allowedRoles[]	Role-based route guard

Navigation uses React Router v6 with **?tab=xxx** query parameters. Sidebar links update the URL query string, and each dashboard reads **useSearchParams()** to activate the correct tab — enabling deep-linking and browser back/forward navigation.

8 Notification System

SupplySync uses a **database-backed, polling-based notification system**. Notifications are created server-side when business events occur and delivered to the frontend via a 15-second polling interval.

Event	Type	Recipients
Vendor registers	VendorRegistration	All Procurement Officers
Vendor approved / rejected	VendorStatusUpdate	The Vendor
Vendor suspended	VendorSuspended	The Vendor
Contract created	ContractCreated	The Vendor
Purchase Order created	POCreated	The Vendor
Goods received (any status)	GoodsReceived	The Vendor
Goods Receipt has issues	GoodsReceiptIssue	Procurement Officer (creator of PO)
GR Accepted — invoice expected	InvoiceSubmitted	All Finance Officers
Invoice submitted by vendor	InvoiceSubmitted	All Finance Officers
Invoice approved / rejected	InvoiceReviewed	The Vendor
Payment processed	PaymentProcessed	The Vendor
Compliance check failed	ComplianceFail	All Admins + All Procurement Officers
Low stock alert (qty ≤ 10)	LowStockAlert	Warehouse Manager who issued the item

9 Security & Authentication

Concern	Implementation
Authentication	JWT Bearer tokens — 8-hour expiry, HMAC-SHA256 signed, issued on login
Token Storage	localStorage (key: 'auth') — parsed on mount by AuthContext
Auto-logout	Axios response interceptor removes token and redirects on 401
Password Policy	Min 6 chars, at least 1 digit (ASP.NET Identity)
Role Enforcement (API)	[Authorize(Roles=...)] on every controller action
Role Enforcement (Frontend)	ProtectedRoute component checks user.role before rendering
CORS	Restricted to http://localhost:5173 (Vite dev server)
Cascade Delete Safety	All FK relationships use OnDelete(NoAction) to prevent accidental cascades
Inactive Users	IsActive flag checked on login — inactive users get 401 Unauthorized
Password Reset	Admin-only via token-based reset (GeneratePasswordResetTokenAsync)

JWT Token Claims

Each generated token contains the following claims:

- ClaimTypes.NameIdentifier → user.Id (used to identify the caller in every service)
- ClaimTypes.Email → user.Email
- ClaimTypes.Name → user.FullName
- ClaimTypes.Role → one entry per role assigned to the user

10 Testing

The project includes a dedicated **SupplySync.Tests** project using **NUnit 4** with **Moq** for mocking. Tests cover all controllers and services with both happy-path and failure scenarios.

Test File	Class Under Test	Method(s) Tested	Test Count
AdminControllerTests	AdminController	GetAllUsers()	2
AuthControllerTests	AuthController	Login()	3
ComplianceControllerTests	ComplianceController	PerformCheck()	2
ContractControllerTests	ContractController	Create()	2
GoodsReceiptControllerTests	GoodsReceiptController	Create()	3
InventoryControllerTests	InventoryController	IssueItem()	3
InvoiceControllerTests	InvoiceController	Review()	3
NotificationControllerTests	NotificationController	MarkAsRead()	3
PurchaseOrderControllerTests	PurchaseOrderController	Create()	2
ReportsControllerTests	ReportsController	ProcurementSpending()	2
VendorControllerTests	VendorController	UpdateStatus()	3
ComplianceServiceTests	ComplianceService	PerformCheckAsync()	3
ContractServiceTests	ContractService	CreateContractAsync()	3
GoodsReceiptServiceTests	GoodsReceiptService	CreateGoodsReceiptAsync()	4
InventoryServiceTests	InventoryService	IssueItemAsync()	4
InvoiceServiceTests	InvoiceService	ReviewInvoiceAsync(), SubmitInvoiceAsync()	5
NotificationServiceTests	NotificationService	MarkAsReadAsync()	3
PurchaseOrderServiceTests	PurchaseOrderService	CreatePOAsync()	4
VendorServiceTests	VendorService	UpdateVendorStatusAsync()	4

Run all tests: `dotnet test --verbosity normal`

Run specific test: `dotnet test --filter "ClassName=InvoiceServiceTests"`

11 Deployment & Configuration

11.1 Backend — appsettings.json

```
{ "ConnectionStrings": { "DefaultConnection":  
"Server=.;Database=SupplySyncDB;Trusted_Connection=True;" }, "Jwt": { "Key":  
"<min-32-char-secret-key>", "Issuer": "SupplySync", "Audience": "SupplySyncUsers" } }
```

11.2 Backend Setup Commands

```
dotnet restore  
dotnet ef database update  
dotnet run  
# API available at http://localhost:5000  
# Swagger UI at http://localhost:5000/swagger
```

11.3 Frontend Setup Commands

```
cd SupplySync_frontend  
npm install  
npm run dev  
# App available at http://localhost:5173
```

11.4 Test Project Setup

```
cd SupplySync.Tests  
dotnet add package NUnit  
dotnet add package NUnit3TestAdapter  
dotnet add package Microsoft.NET.Test.Sdk  
dotnet add package Moq  
dotnet add package Microsoft.EntityFrameworkCore.InMemory  
dotnet test
```

11.5 Environment Notes

Setting	Default	Notes
API Port	5000	Configurable in launchSettings.json
Frontend Port	5173	Vite default — must match CORS origin in API
CORS Origin	http://localhost:5173	Set in Program.cs AddCors policy
JWT Expiry	8 hours	Configurable in JwtHelper.cs
Low Stock Threshold	10 units	Hardcoded in InventoryService.cs
Notification Poll Interval	15 seconds	Configurable in NotificationContext.tsx
DB Seed Roles	6 roles auto-seeded on startup	Program.cs seeds roles if not present

SupplySync Technologies Pvt. Ltd. | Chennai, Tamil Nadu, India | support@supplysync.io | www.supplysync.io